

OpenAI Assistants API

True Server-Side Stateful Conversation — Implementation Guide

Audience: Fresher / Entry-Level

API: OpenAI Assistants API (Beta)

Platform: AWS EC2 Ubuntu + code-server

1. How the Assistants API Differs from Chat Completions

In the Chat Completions API, every call is stateless. Your code must manually maintain a messages list and re-send the system prompt and full history on every call. The Assistants API introduces three server-side objects — Assistant, Thread, and Run — that move this responsibility to OpenAI's servers.

Aspect	Chat Completions API	Assistants API (this demo)
System prompt	Re-sent on EVERY call by your code	Uploaded ONCE when Assistant is created
Conversation history	Your code builds and sends it every call	Stored server-side in a Thread automatically
What you send per turn	system + full history + new message	ONLY the new user message
State management	Your responsibility (HistoryManager class)	OpenAI's responsibility
Response style	Synchronous — reply returned immediately	Async — submit Run, poll for completion
Cleanup needed?	No — stateless	Yes — delete Thread and Assistant when done

2. The Three Server-Side Objects

Assistant

A persistent object on OpenAI servers that holds the system prompt (called 'instructions'), the model name, and any tools. Created once per deployment. Has a permanent ID like `asst_abc123`. You reuse this ID for every conversation — the system prompt never leaves OpenAI's side again.

Thread

A server-side conversation container. Stores every user and assistant message automatically. One Thread = one conversation session. You never manage a messages list in your code. To start a new topic, create a new Thread — the Assistant stays the same.

Run

The execution unit. When you trigger a Run, OpenAI combines [Assistant instructions + Thread messages] and sends them to the model. Runs are asynchronous — you submit and poll for status: `queued` → `in_progress` → `completed`. Only after `completed` can you fetch the reply.

3. Project Folder Structure

```
# Project Layout
assistants_demo/
|
|-- main.py                <- Entry point. Setup, loop, teardown.
|-- requirements.txt       <- openai>=1.0.0
|
|-- config/
|   |-- __init__.py
|   +-- settings.py       <- API key, model, SYSTEM_PROMPT.
|
|-- core/
|   |-- __init__.py
|   |-- assistant_manager.py <- Creates/deletes the Assistant (uploads system prompt).
|   |-- thread_manager.py   <- Creates Thread, adds messages, fetches replies.
|   |-- run_manager.py      <- Submits Run and polls until completed.
|   +-- chatbot.py         <- Orchestrates all three managers per turn.
|
+-- utils/
    |-- __init__.py
    +-- display.py         <- All terminal print helpers.
```

File Responsibilities:

config/settings.py	SYSTEM_PROMPT, API key, model, polling config. Single source of truth.
core/assistant_manager.py	create(): uploads system prompt to OpenAI once. delete(): cleans up.
core/thread_manager.py	create(): new conversation container. add_user_message(): one message only. get_latest_reply(): fetches reply.
core/run_manager.py	execute(): submits Run, polls status every second, returns usage stats.

core/chatbot.py	setup(), ask(), new_conversation(), teardown(). Ties all three managers together.
utils/display.py	All print helpers: banner, setup info, reply, history, token summary.
main.py	Input loop only. Calls bot.setup(), bot.ask(), display helpers, bot.teardown().

4. Exact Data Flow — What Travels Over the Network

★ This is the most important section. Study what is sent to OpenAI at each stage compared to the Chat Completions approach.

Setup Phase (runs ONCE at the start):

1

AssistantManager.create() — Network call to OpenAI

Sends: SYSTEM_PROMPT + MODEL_NAME to POST /v1/assistants

OpenAI stores this and returns an assistant_id.

Your system prompt now lives on OpenAI's servers permanently.

You never send the system prompt again for the entire session.

2

ThreadManager.create() — Network call to OpenAI

Sends: POST /v1/threads (empty body)

OpenAI creates an empty conversation container and returns a thread_id.

This is where all future messages will be stored server-side.

Per-Turn Phase (repeats for every user message):

3

ThreadManager.add_user_message(text) — Network call

Sends: ONLY the new user message text to POST /v1/threads/{id}/messages

Nothing else. No system prompt. No history. Just the new text.

OpenAI appends it to the Thread.

4

RunManager.execute() — Submit + Poll

Sends: assistant_id + thread_id to POST /v1/threads/{id}/runs

OpenAI internally combines: [Assistant instructions] + [all Thread messages]

and sends that full context to the model. You never see or send this bundle.

Then polls GET /v1/threads/{id}/runs/{run_id} every second until status=completed.

5

ThreadManager.get_latest_reply() — Fetch reply

Sends: GET /v1/threads/{id}/messages

OpenAI returns the latest assistant message from the Thread.

The reply was already stored in the Thread by the Run.

Teardown Phase (runs ONCE at the end):

6

bot.teardown() — Cleanup

Deletes the Thread: DELETE /v1/threads/{id}

Deletes the Assistant: DELETE /v1/assistants/{id}

Always do this — orphaned Assistants accumulate on your OpenAI account.

5. Step-by-Step Implementation Guide

Step A — Create Project on AWS EC2 Ubuntu

```
# Terminal

mkdir ~/assistants_demo && cd ~/assistants_demo

mkdir config core utils

touch config/__init__.py core/__init__.py utils/__init__.py
```

Step B — Install Dependency

```
# Terminal

pip install openai

python -c "import openai; print(openai.__version__)"

# Must be >= 1.0.0 for Assistants API support
```

Step C — config/settings.py

Set your API key and system prompt here. Nothing else needs to change.

```
# config/settings.py

OPENAI_API_KEY    = "sk-proj-XXXX"    # Your key here

MODEL_NAME        = "gpt-4o-mini"

ASSISTANT_NAME    = "DataBot"

POLL_INTERVAL_SECONDS = 1

MAX_POLL_ATTEMPTS    = 30

SYSTEM_PROMPT = """

You are DataBot, a Data Engineering tutor for freshers.

Give concise 3-5 sentence answers. Stay on topic.

"""
```

Step D — core/assistant_manager.py

Creates the Assistant on OpenAI's servers using the system prompt from settings. This is the ONLY place the system prompt is ever transmitted.

```
# core/assistant_manager.py (create method)

def create(self) -> str:

    self.assistant = self._client.beta.assistants.create(

        name          = ASSISTANT_NAME,

        instructions   = SYSTEM_PROMPT, # <- sent to OpenAI ONCE

        model          = MODEL_NAME,
```

```

)

self.assistant_id = self.assistant.id

return self.assistant_id

```

Step E — core/thread_manager.py

The `add_user_message()` method is the core of the token saving. It sends nothing but the new text.

```

# core/thread_manager.py (add_user_message)

def add_user_message(self, content: str) -> None:

    self._client.beta.threads.messages.create(

        thread_id = self.thread_id,

        role      = "user",

        content   = content,    # <- ONLY this is sent per turn

    )

    # No system prompt. No history. OpenAI already has both.

```

Step F — core/run_manager.py

Submits a Run and polls until completed. The Run is what triggers OpenAI to combine the system prompt + thread history + new message and generate a reply.

```

# core/run_manager.py (execute method – simplified)

def execute(self, thread_id, assistant_id):

    run = self._client.beta.threads.runs.create(

        thread_id    = thread_id,

        assistant_id = assistant_id,

    )

    while True:

        time.sleep(1)

        run = self._client.beta.threads.runs.retrieve(

            thread_id=thread_id, run_id=run.id)

        if run.status == "completed":

            return {"status": "completed", "usage": run.usage}

        if run.status in ("failed", "expired"):

            return {"status": run.status}

```

Step G — core/chatbot.py

```

# core/chatbot.py (ask method)

def ask(self, user_input: str):

    # Step 1: Add ONLY the new message to Thread

```

```

self._thread_mgr.add_user_message(user_input)

# Step 2: Trigger Run (OpenAI handles system+history internally)
result = self._run_mgr.execute(self.thread_id, self.assistant_id)

# Step 3: Fetch the reply OpenAI added to the Thread
reply = self._thread_mgr.get_latest_reply()

return reply, result["usage"]

```

Step H — main.py

```

# main.py (simplified)

bot = Chatbot()

bot.setup()          # Create Assistant + Thread on OpenAI servers

while True:
    user_input = input("You: ")
    if user_input == "quit":
        bot.teardown() # Delete Thread + Assistant from OpenAI
        break
    reply, usage = bot.ask(user_input) # Only new message sent
    print(f"DataBot: {reply}")

```

Step I — Run the Demo

```

# Terminal

cd ~/assistants_demo

python main.py

# Expected output:

# Setting up Assistant and Thread on OpenAI servers...
# [SETUP] Assistant created: asst_abc123
#           Thread created:      thread_xyz789
#           System prompt uploaded once. Will NOT be re-sent per turn.

# You: what is apache kafka
# [Sending message to Thread... waiting for Run to complete]
# DataBot: Apache Kafka is a distributed event streaming platform...
# [Turn 1 | Prompt: 298 tok | Completion: 74 tok | Total: 372 tok]

# You: history    <- fetches full Thread from OpenAI
# You: new        <- creates new Thread, same Assistant
# You: quit       <- deletes Thread and Assistant, shows summary

```

6. Architecture — What Lives Where

```
# Object Ownership Map
```

YOUR CODE (local)		OPENAI SERVERS
config/settings.py		Assistant object
SYSTEM_PROMPT	uploaded	instructions = SYSTEM_PROMPT
MODEL_NAME	once	model = gpt-4o-mini id = asst_abc123
core/thread_manager.py		Thread object
add_user_message(text)		messages = [sends only: text {role:user, content:"Q1"}, {role:assistant,content:"A1"}, {role:user, content:"Q2"}, {role:assistant,content:"A2"}, ...]
core/run_manager.py		Run object
execute(thread_id, asst_id)		OpenAI combines: [Assistant.instructions] + [Thread.messages] → sends to model → appends reply to Thread → status = completed

7. Important Note on Token Billing

! OpenAI still bills you for the system prompt tokens on every Run. The saving is that YOU do not write code to re-send it — OpenAI includes it internally. The billing model is identical to Chat Completions, but your code is simpler and there is zero risk of accidental duplication.

What	Chat Completions	Assistants API
System prompt in your code	In messages list, re-sent manually	Uploaded once, never in your code again
System prompt tokens billed	Every call	Every Run (OpenAI includes it internally)

Risk of duplication	Yes — if developer appends to history	None — developer cannot touch it
Code complexity	You manage history list	OpenAI manages everything

8. Common Errors and Fixes

Error: AuthenticationError

Fix: Wrong API key. Update OPENAI_API_KEY in config/settings.py.

Error: Run status: failed

Fix: Usually a model error. Check MAX_POLL_ATTEMPTS in settings. Try a different model name like gpt-3.5-turbo.

Error: Run status: timeout

Fix: Increase MAX_POLL_ATTEMPTS in settings.py. Network latency can add delay.

Error: NotFoundError on thread or assistant

Fix: Orphaned IDs from a previous crashed session. Run bot.teardown() manually or delete via OpenAI platform dashboard.

Error: Reply is '(no reply)' after completed Run

Fix: The Run completed but the reply was not yet indexed. Add a small `time.sleep(0.5)` before `get_latest_reply()`.